

Scalable AI

Retrieval Augmentation

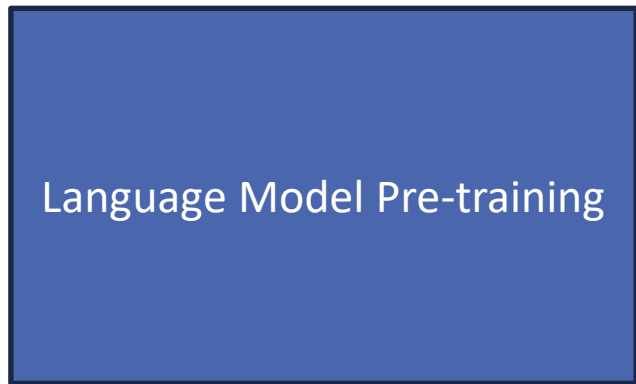
Lorenzo Baraldi

Outline

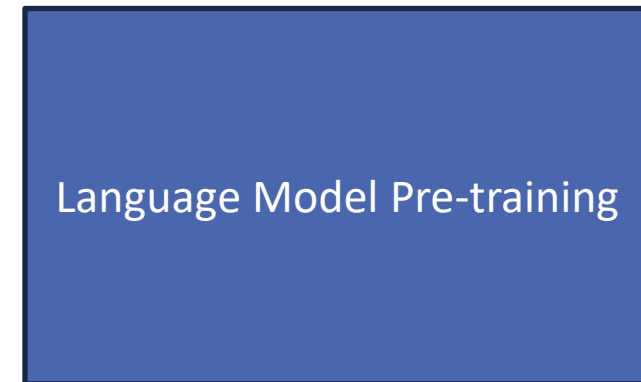
- Pre-training language models with the help of Retrieval
 - REALM
 - RETRO
- Jointly fine-tuning a retriever and language model for downstream task
 - RAG for Knowledge-Intensive NLP Tasks
 - REPLUG
- More comprehensive Interaction between language model and retriever
 - Self-RAG

What is RAG

- Memorize World Knowledge in Model Parameters VS Store World Knowledge in an External Database



World Knowledge (Corpus)



World Knowledge (Corpus)

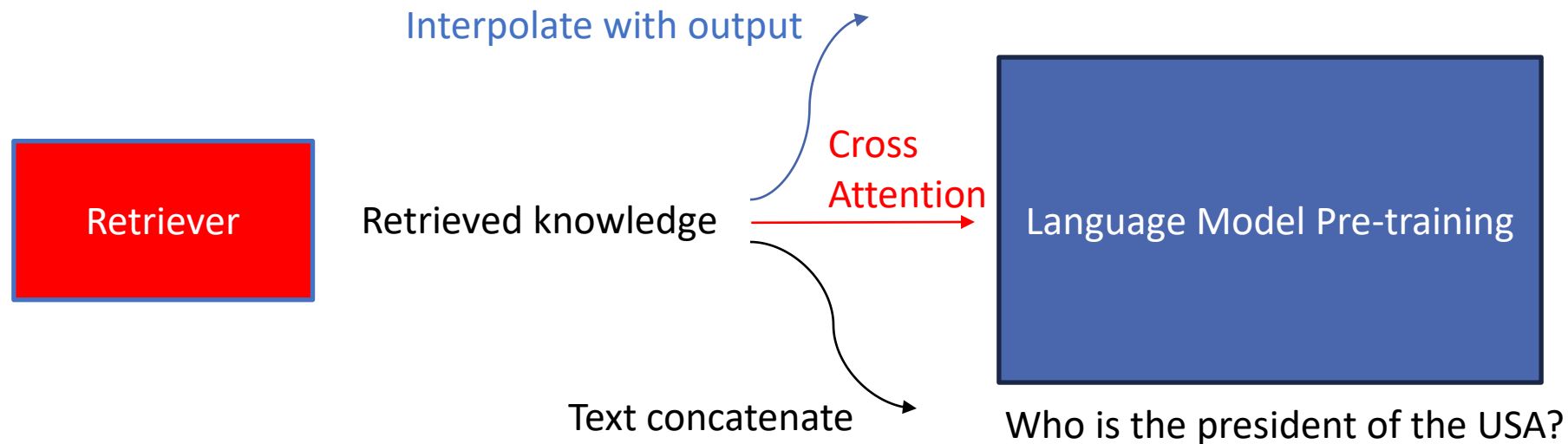


Why RAG

- Explainability
 - LLM's output comes from retrieved knowledge rather than its internal memory
- Privacy
 - Use proprietary data as external knowledge rather than training data
- Decompose memorization and other capability in pre-training
 - LLM can focus on the tasks other than memorization
 - Small LLM + Retriever leads to better performance than solely Large LLM

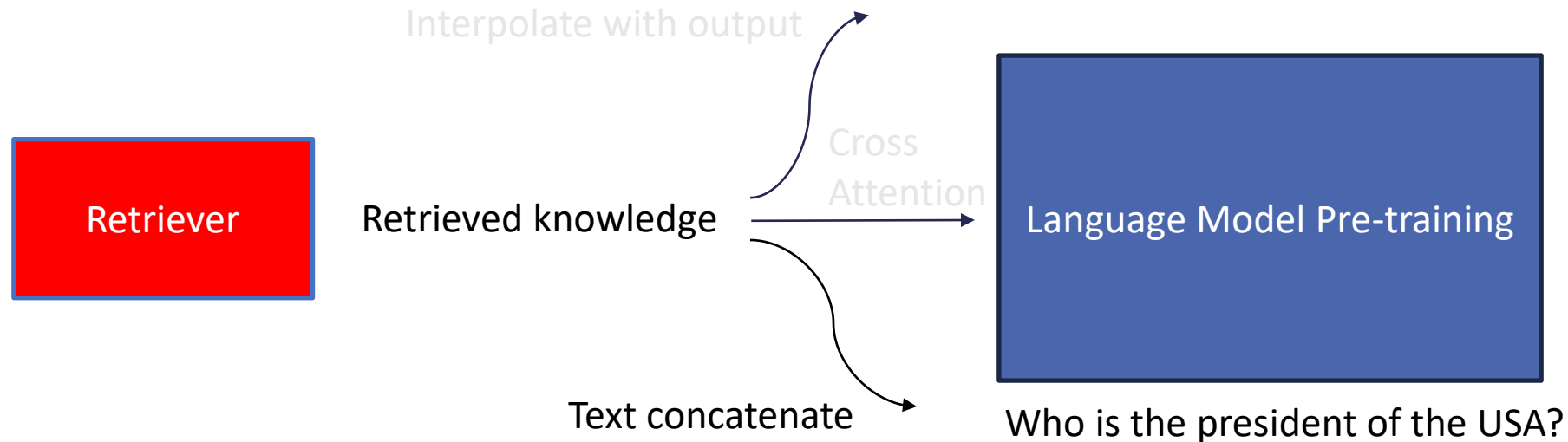
Different Types of RAG

- How to fuse retrieved knowledge with input text (e.g., Question)
 - Insert retrieved knowledge by text concatenation (e.g., REALM)
 - Insert retrieved knowledge by cross attention (e.g., RETRO, FID)
 - Insert retrieved knowledge by interpolate language model output probability (e.g., KNNLM)



Different Types of RAG

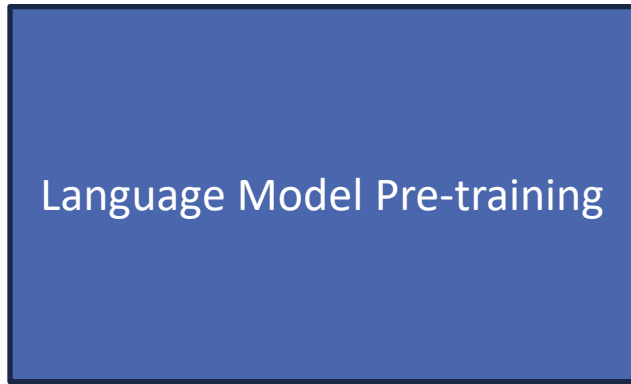
- How to fuse retrieved knowledge with input text (e.g., Question)
 - Insert retrieved knowledge by text concatenation (e.g., REALM)
 - Insert retrieved knowledge by cross attention (e.g., Retro, FID)
 - Insert retrieved knowledge by interpolate language model output probability (e.g., KNNLM)



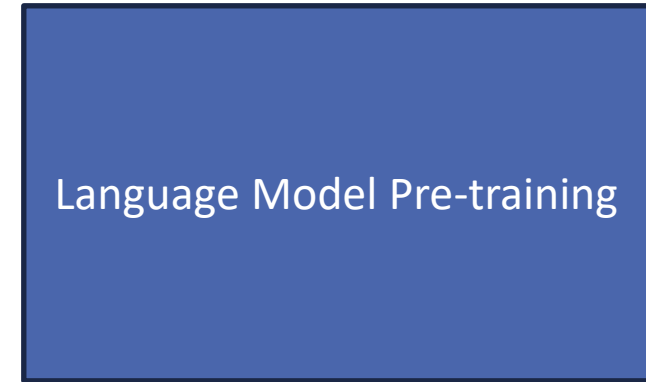
Outline

- Pre-training language models with the help of Retrieval
 - REALM
 - RETRO
- Jointly fine-tuning a retriever and language model for downstream task
 - RAG for Knowledge-Intensive NLP Tasks
 - REPLUG
- More comprehensive Interaction between language model and retriever
 - Self-RAG

REALM: Illustration of RAG Pre-Training



the at the top of the [MASK] pyramid



the at the top of the [MASK] pyramid



The pyramidion on top allows for
less material higher up the pyramid



the at the top of the [MASK] pyramid

REALM: Application to Downstream Tasks

Pre-training task: Input: Concatenate retrieved passages and masked text --> Output: masked token prediction

Fine-tuning task: Input: Concatenate retrieved passages and query --> Output: Answer span in the retrieved passages

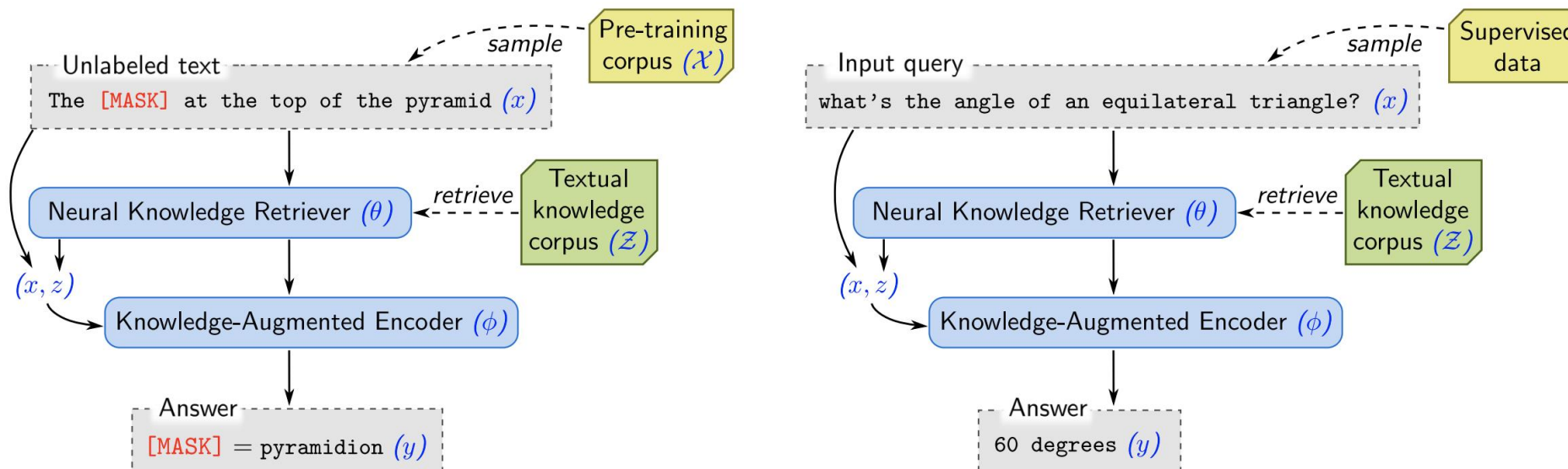


Figure 2. The overall framework of REALM. **Left:** *Unsupervised pre-training.* The knowledge retriever and knowledge-augmented encoder are jointly pre-trained on the unsupervised language modeling task. **Right:** *Supervised fine-tuning.* After the parameters of the retriever (θ) and encoder (ϕ) have been pre-trained, they are then fine-tuned on a task of primary interest, using supervised examples.

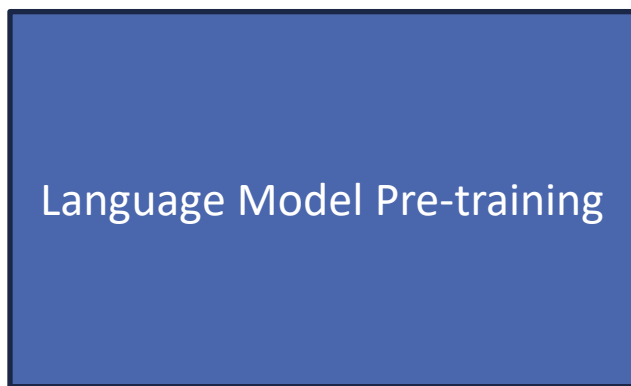
REALM: Challenges of RAG Pre-Training

- How to Update Retriever in MLM Pre-Training
- Synchronized Problem when Updating Retriever
- How to Create Training Data in RAG MLM Pre-Training



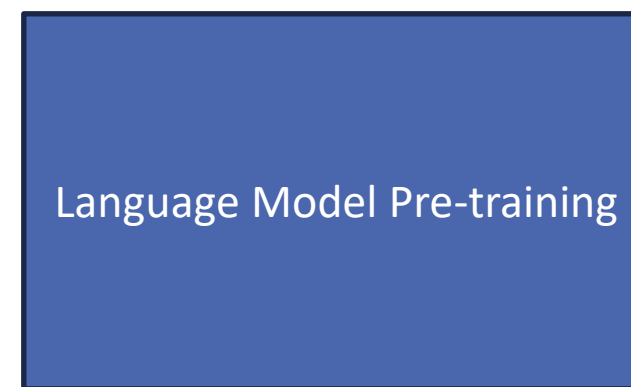
REALM: How to Update Retriever?

$$p(y \mid \boxed{x})$$



the at the top of the [MASK] pyramid

$$p(y \mid x) = \sum_{z \in \mathcal{Z}} p(y \mid \boxed{x}, \boxed{z}) p(\boxed{z} \mid \boxed{x})$$



the at the top of the [MASK] pyramid



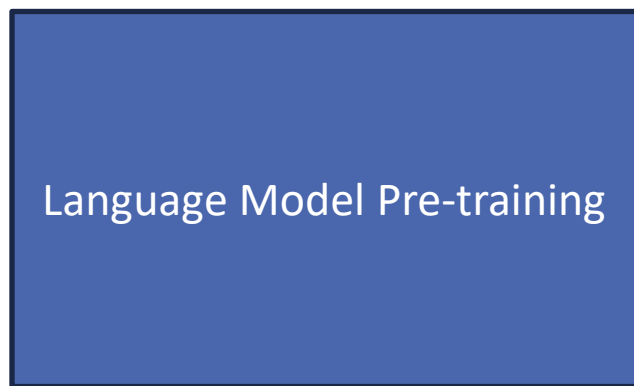
The pyramidion on top allows for less material higher up the pyramid



the at the top of the [MASK] pyramid

REALM: How to Update Retriever?

$$P(y|x, z_1)$$



the at the top of the [MASK] pyramid

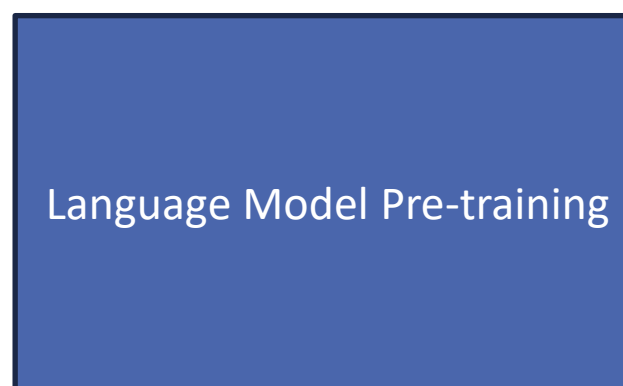
$$P(z_1|x)$$

The pyramidion on top allows for less material higher up the pyramid

Retriever

the at the top of the [MASK] pyramid

$$P(y|x, z_2)$$



the at the top of the [MASK] pyramid

$$P(z_2|x)$$

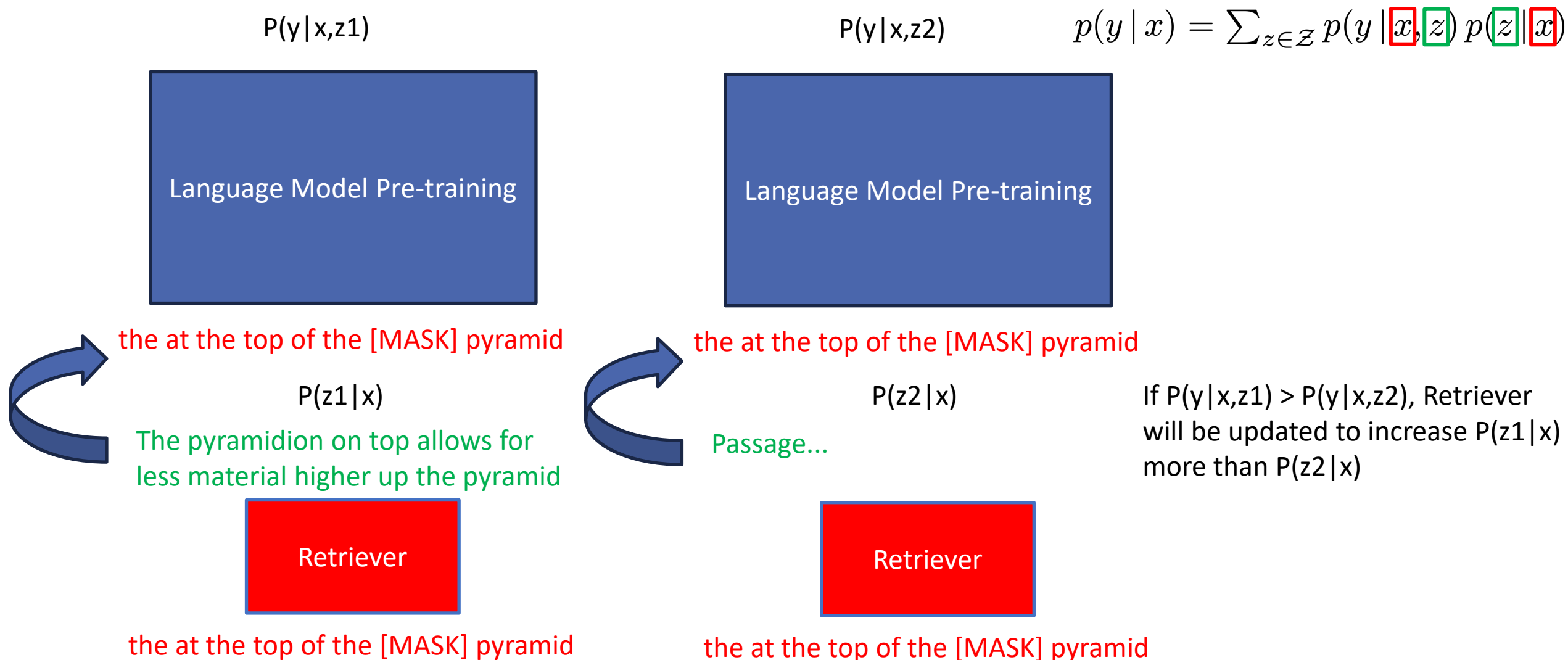
Passage...

Retriever

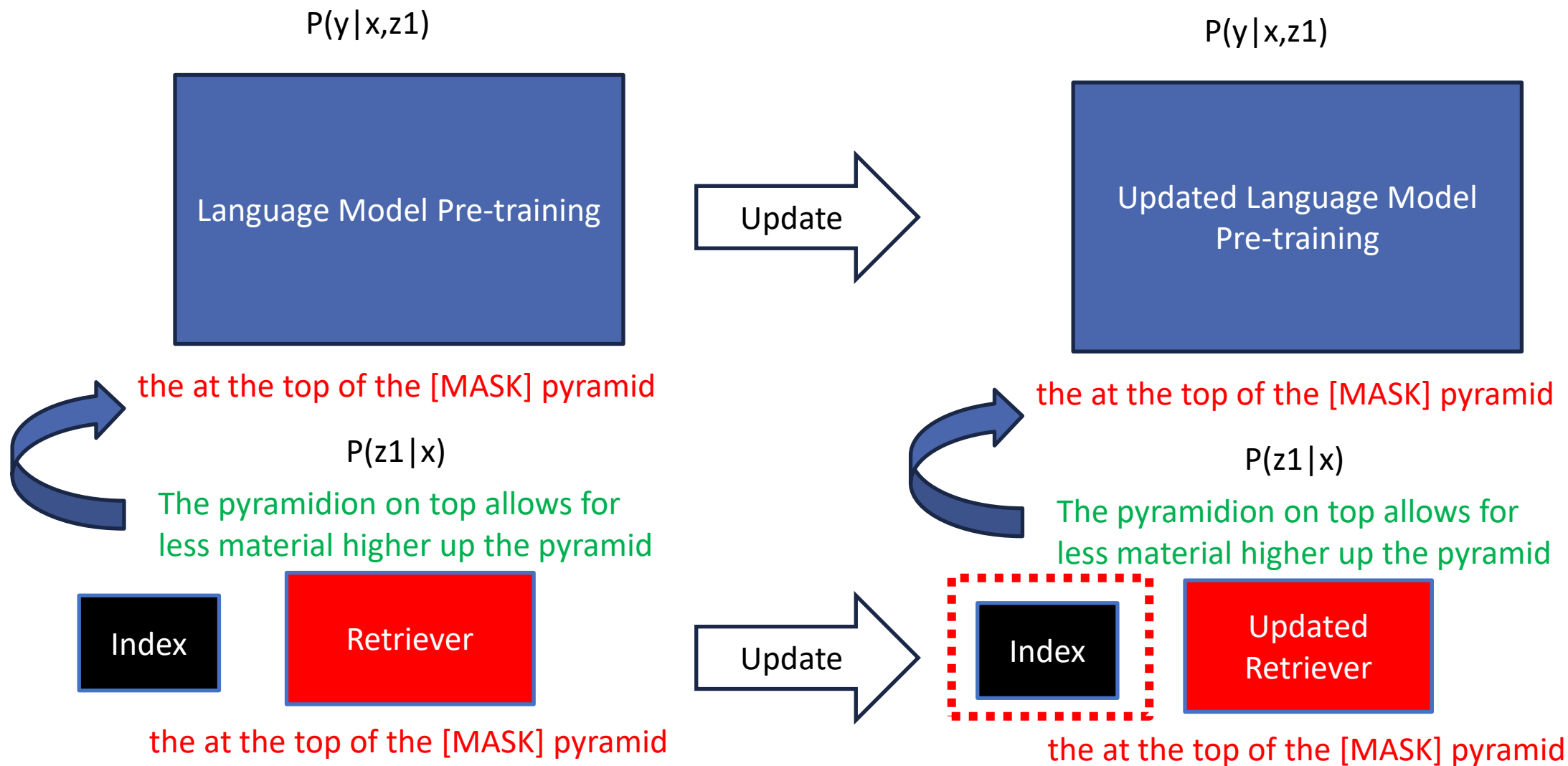
the at the top of the [MASK] pyramid

$$p(y|x) = \sum_{z \in \mathcal{Z}} p(y|\boxed{x}, \boxed{z}) p(\boxed{z}|\boxed{x})$$

REALM: How to Update Retriever?

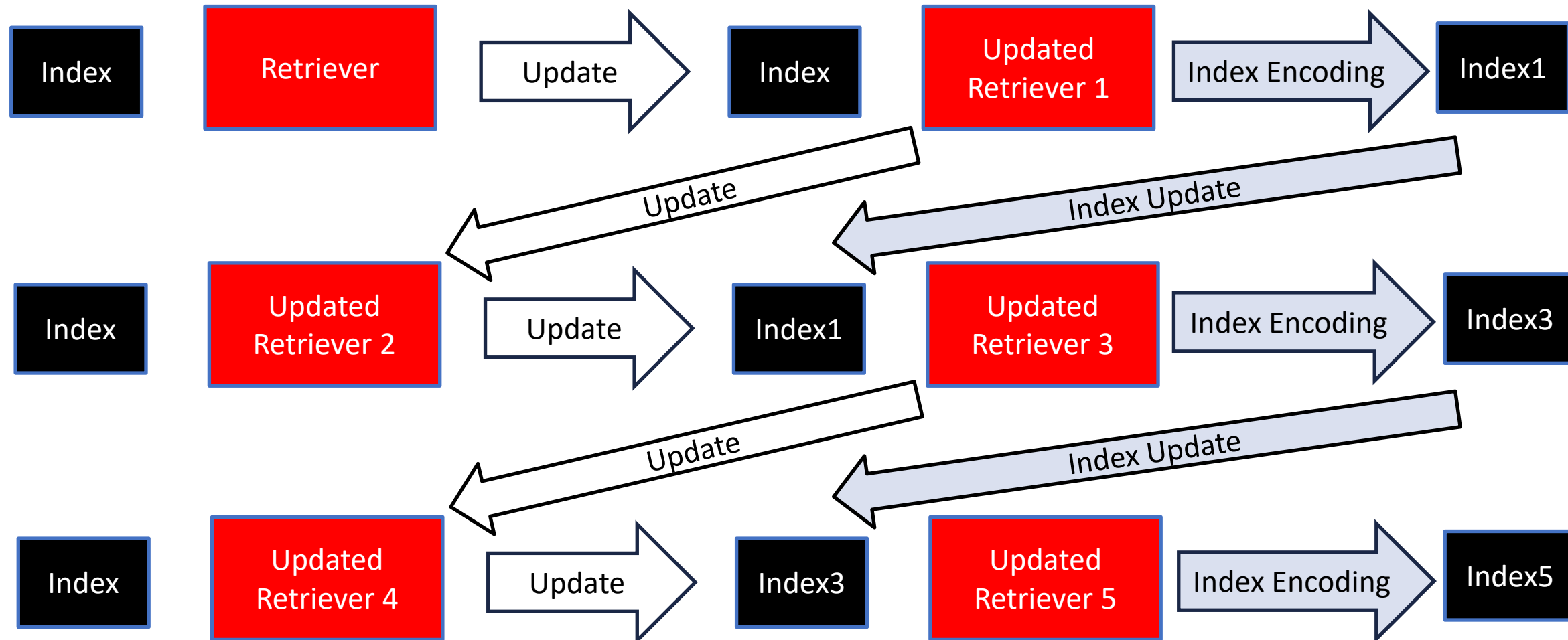


REALM: Solution to Synchronized Issue



REALM: Asynchronous Index Update

Assuming it requires two update times for a Retriever to encode index



REALM: Data for RAG MLM Pre-Training

- Salient Span Masking: Name Entity (e.g., United Kingdom) or Dates (e.g., July 9)
- Null Documents: Replacing Retrieved Documents with Null so that Model can only rely on its knowledge to predict [MASK]
- Prohibiting Trivial Retrieval: If the masked sentence comes from the retrieved document; the task becomes too trivial. Thus, the document is removed from the top-K documents.

REALM: Effectiveness Comparison

Table 1. Test results on Open-QA benchmarks. The number of train/test examples are shown in parentheses below each benchmark. Predictions are evaluated with exact match against any reference answer. Sparse retrieval denotes methods that use sparse features such as TF-IDF and BM25. Our model, REALM, outperforms all existing systems.

Name	Architectures	Pre-training	NQ (79k/4k)	WQ (3k/2k)	CT (1k /1k)	# params
BERT-Baseline (Lee et al., 2019)	Sparse Retr.+Transformer	BERT	26.5	17.7	21.3	110m
T5 (base) (Roberts et al., 2020)	Transformer Seq2Seq	T5 (Multitask)	27.0	29.1	-	223m
T5 (large) (Roberts et al., 2020)	Transformer Seq2Seq	T5 (Multitask)	29.8	32.2	-	738m
T5 (11b) (Roberts et al., 2020)	Transformer Seq2Seq	T5 (Multitask)	34.5	37.4	-	11318m
DrQA (Chen et al., 2017)	Sparse Retr.+DocReader	N/A	-	20.7	25.7	34m
HardEM (Min et al., 2019a)	Sparse Retr.+Transformer	BERT	28.1	-	-	110m
GraphRetriever (Min et al., 2019b)	GraphRetriever+Transformer	BERT	31.8	31.6	-	110m
PathRetriever (Asai et al., 2019)	PathRetriever+Transformer	MLM	32.6	-	-	110m
ORQA (Lee et al., 2019)	Dense Retr.+Transformer	ICT+BERT	33.3	36.4	30.1	330m
Ours ($\mathcal{X}$ = Wikipedia, $\mathcal{Z}$ = Wikipedia)	Dense Retr.+Transformer	REALM	39.2	40.2	46.8	330m
Ours ($\mathcal{X}$ = CC-News, $\mathcal{Z}$ = Wikipedia)	Dense Retr.+Transformer	REALM	40.4	40.7	42.9	330m

- Retrieval + language modeling jointly pre-training improves RAG performance on downstream in-domain tasks (NQ), out-of-domain tasks (WQ, CT)

Other Recent RAG Pre-Training Works

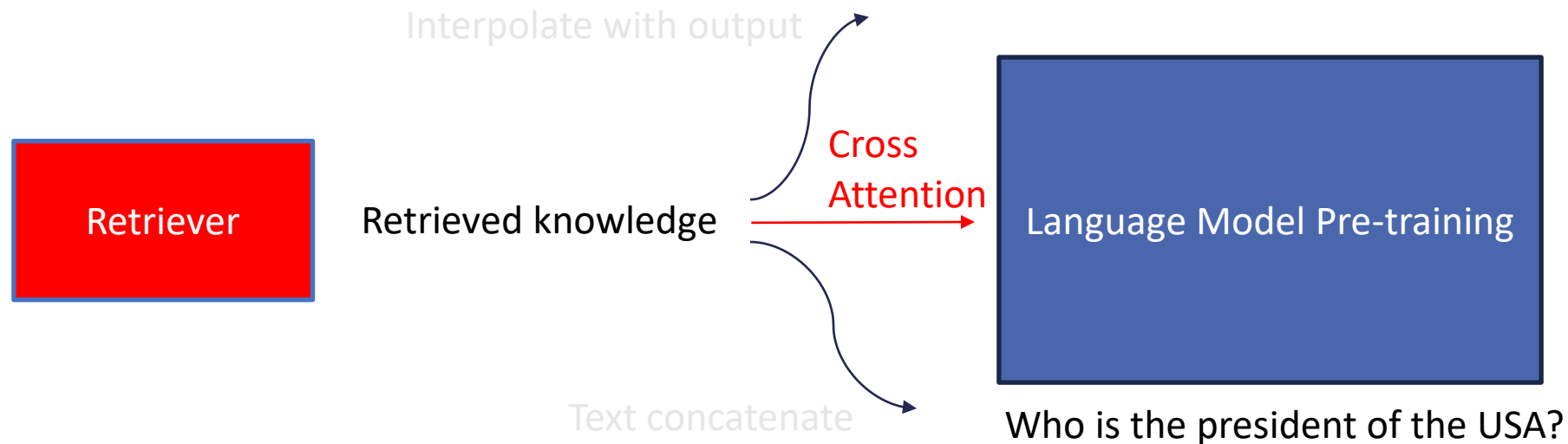
- Atlas: Few-shot Learning with Retrieval Augmented Language Models
 - Encoder-Decoder Structure
 - Start with a better unsupervised pre-trained retriever (i.e., Contriever)
 - Discuss how to update retriever using the signal from language model
 - Discuss more alternatives to unsynchronized index update
- In-Context Pre-Training: Language Model Beyond Document Boundaries
 - Decoder-only structure
 - Aggregate relevant documents to create context input for next word prediction

Outline

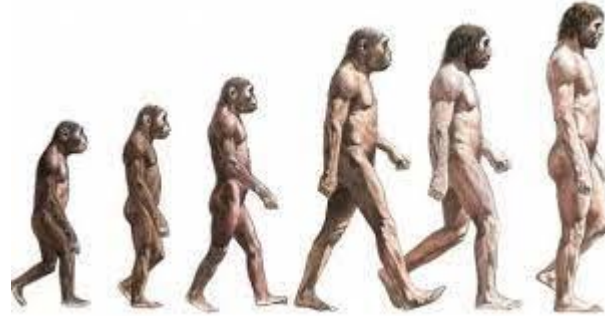
- Pre-training language models with the help of Retrieval
 - REALM
 - **RETRO**
- Jointly fine-tuning a retriever and language model for downstream task
 - **RAG for Knowledge-Intensive NLP Tasks**
 - REPLUG
- More comprehensive Interaction between language model and retriever
 - Self-RAG

Different Types of RAG

- How to Combine Retrieved Knowledge with Input Text (e.g., Question)
 - Insert retrieved knowledge by text concatenation (e.g., REALM)
 - Insert retrieved knowledge by cross attention (e.g., Retro, FID)
 - Insert retrieved knowledge by interpolate language model output probability (e.g., KNNLM)



Why Retro: Scaling Index VS Model Size?



GPT1

GPT2

GPT3

Index

Language
Model Pre-
training

Index

Language
Model Pre-
training

Index

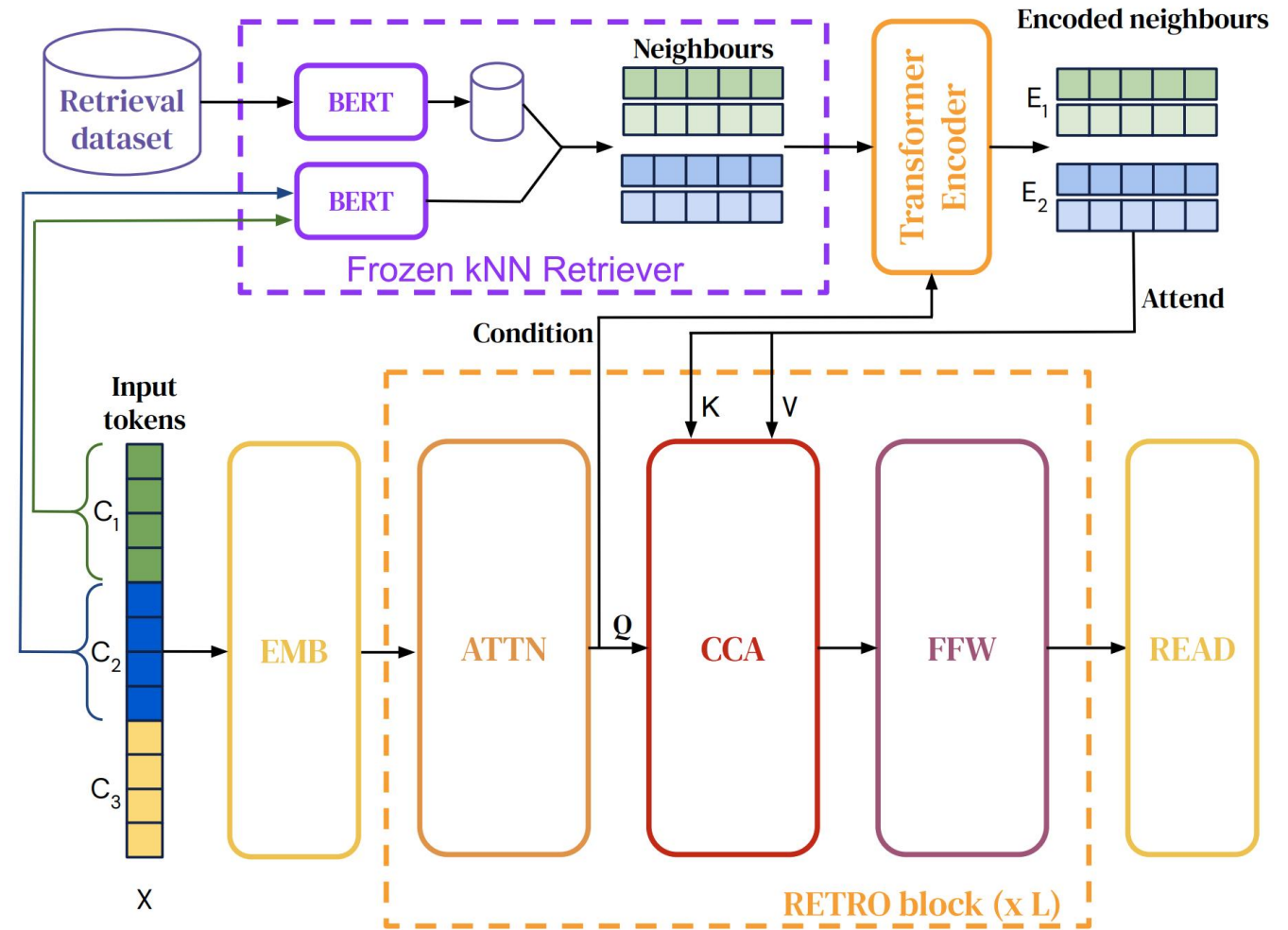
Language
Model Pre-
training

Why Retro: Other Advantages

- Frozen Retriever Parameters: No Synchronized Index Issue
- Chunked Cross-Attention to incorporate retrieved contexts: Linear Time Complexity vs Retrieved Context Length

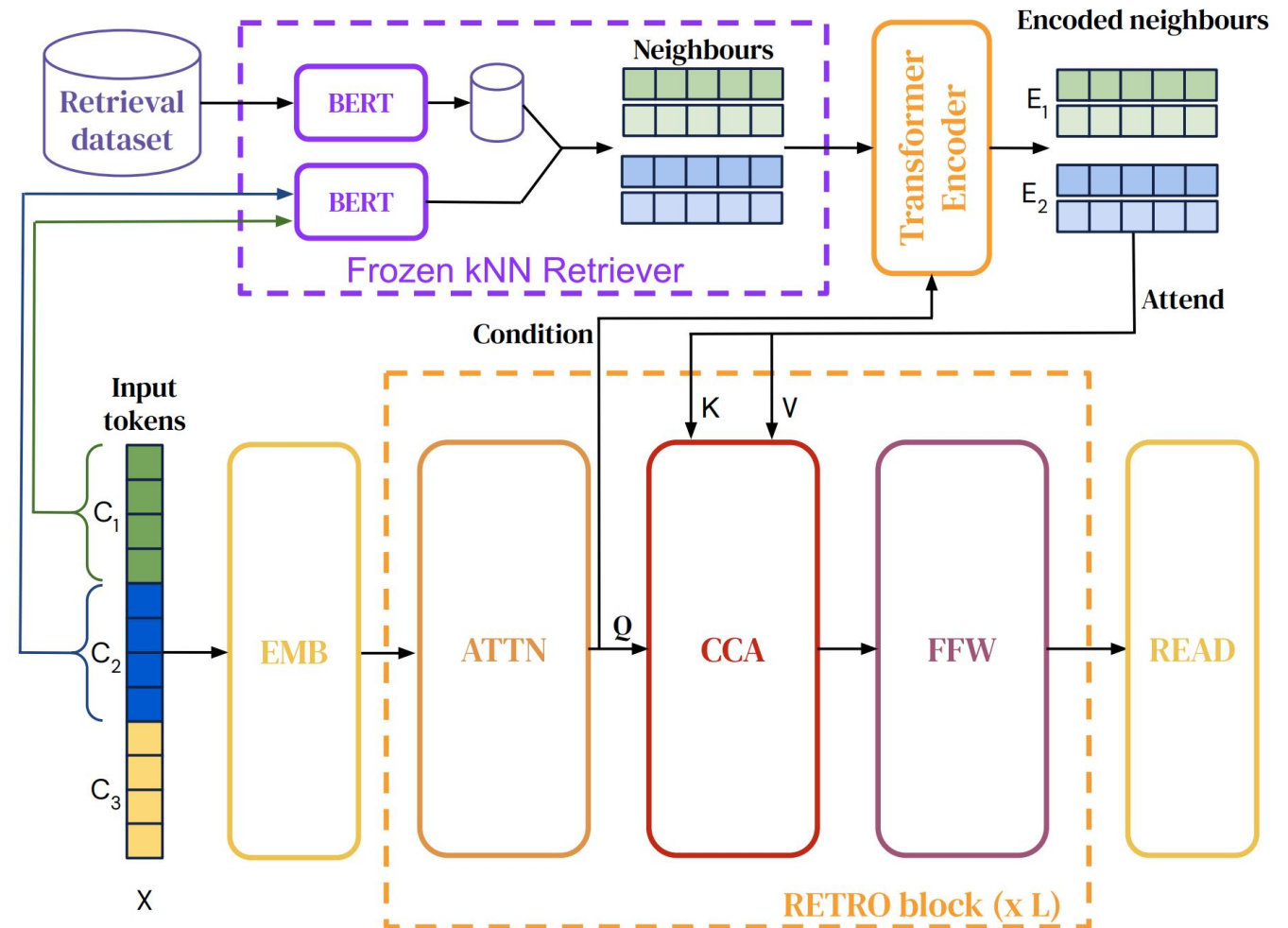
Retro Architecture

- Chunked Retrieval
- Retrieved Chunk Encoding
- Fuse Input and Retrieved Context



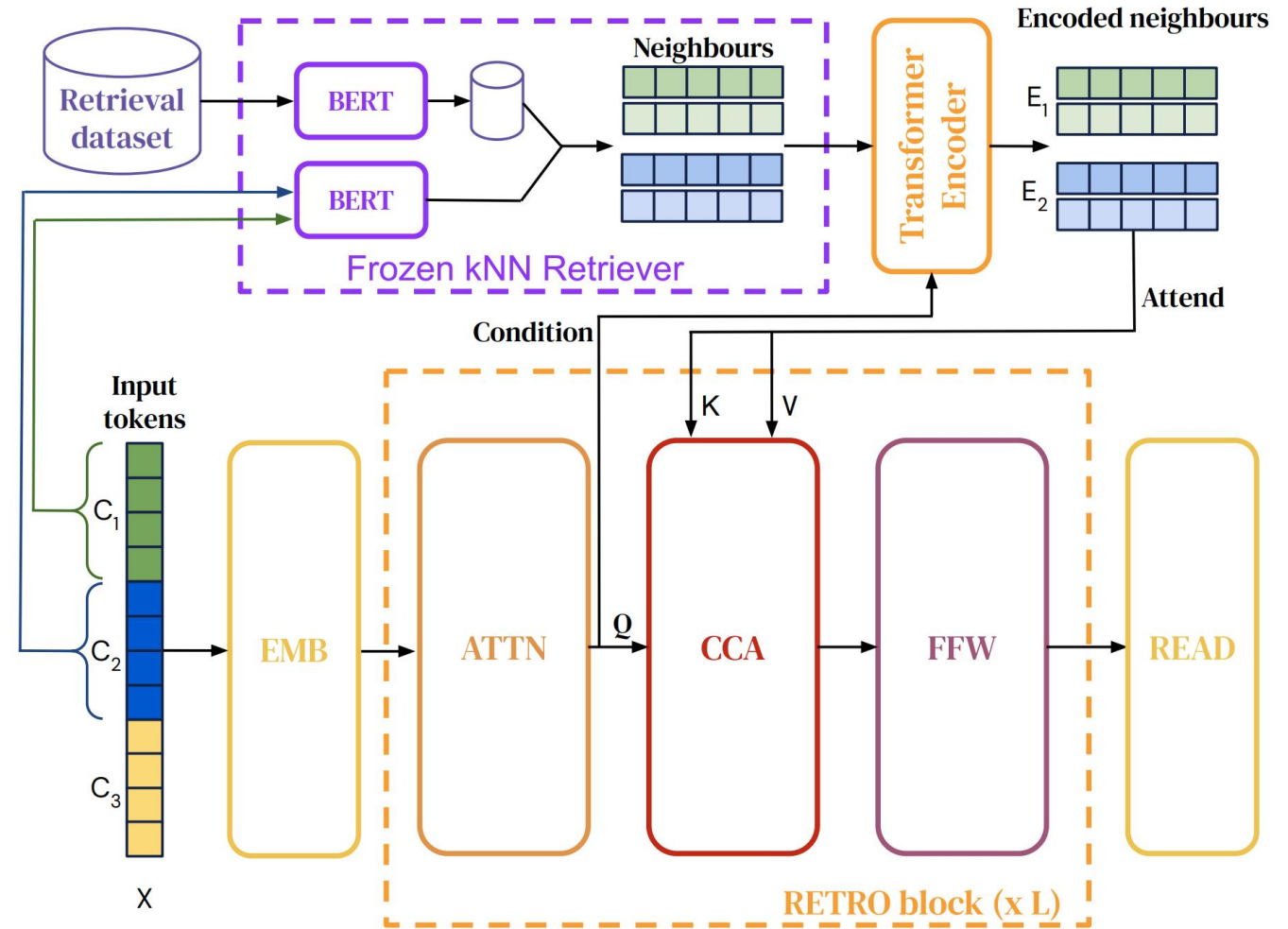
Chunked Retrieval

- Chunked Retrieval:
 - Use frozen BERT retriever
 - Each chunk retrieves its K neighbors plus the neighbors' continuation
 - **Chunked input: Who is the president of the USA**
 - Retrieved chunk: Here is the list of president of the USA.
 - Retrieved chunk continuation: 2008—2016 Barack Obama, 2016—2020 Donald Trump, 2020—2024 Joe Biden.



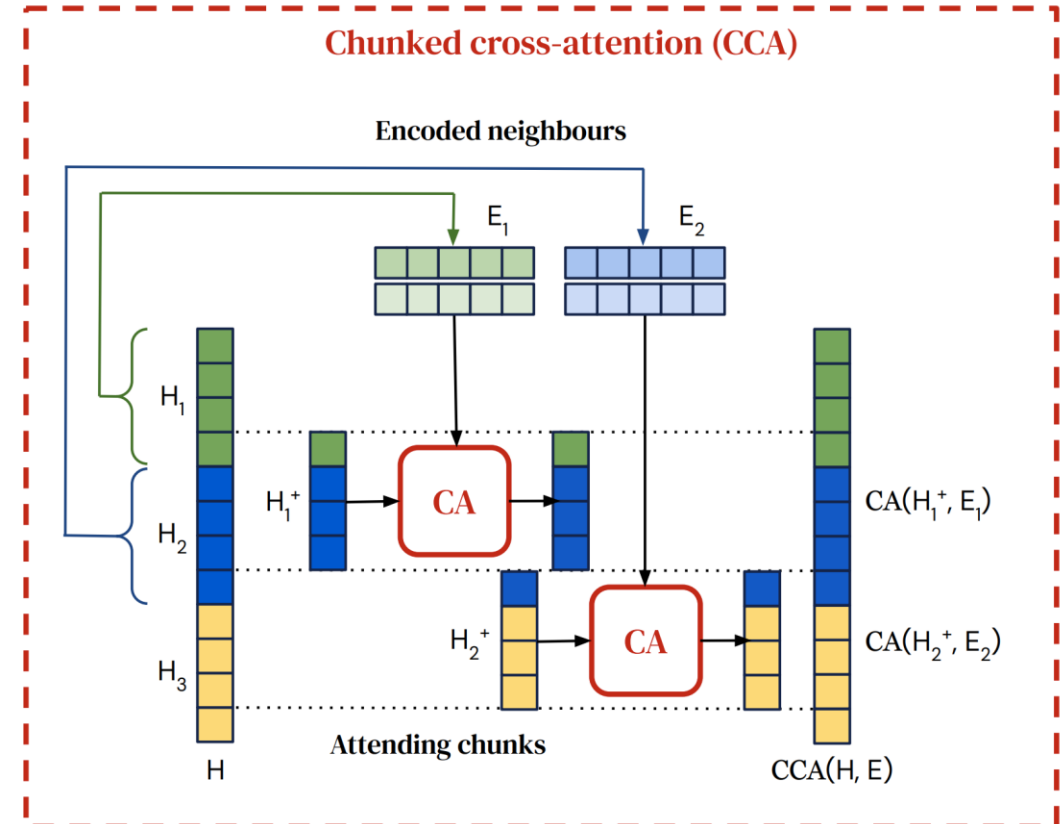
Retrieved Chunk Encoding

- Retrieved Chunk Encoding
 - Trainable Bi-directional Encoder
 - Encoding Retrieved Chunks condition on its query

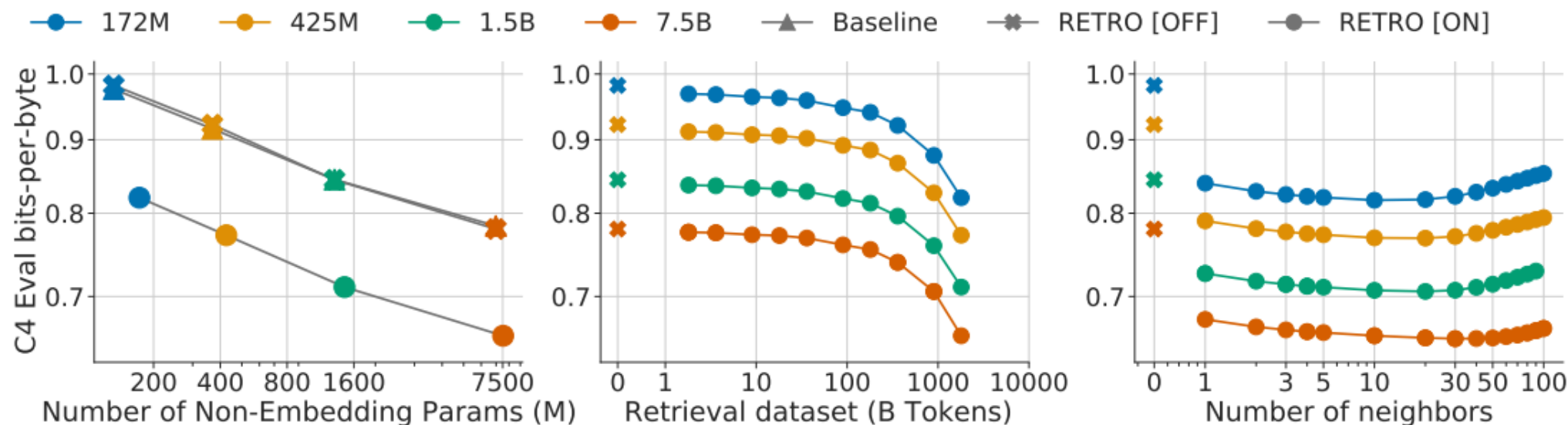


Chunked Cross Attention

- Fusing Input and Retrieved Context with Chunked Cross-Attention
 - Each input chunk only attends the context retrieved by its previous input chunk
 - Like causal mask
 - Linear time complexity with increasing context length
 - Long dependency between chunks can still propagate through self-attention



Retro Results: Language Modelling



- The performance gain of Retro is comparable to multiplying the parameteric model size by $\sim 10\times$
- The gain increases with the size of the retrieval database (middle) and the number of retrieved neighbours

Retro Results: Question Answering



Table 5 | **Question answering results.** Exact match accuracy on Natural Questions.

Model	Test Accuracy
REALM (Guu et al., 2020)	40.4
DPR (Karpukhin et al., 2020)	41.5
RAG (Lewis et al., 2020)	44.5
EMDR ² (Sachan et al., 2021)	52.5
FiD (Izacard and Grave, 2021)	51.4
FiD + Distill. (Izacard et al., 2020)	54.7
Baseline 7B (closed book)	30.4
RETRO 7.5B (DPR retrieval)	45.5

- The performance of Retro is competitive with previous approaches such as Realm, RAG and DPR, but underperforms the more recent FiD.

Outline

- Pre-training language models with the help of Retrieval
 - REALM
 - RETRO
- Jointly fine-tuning a retriever and language model for downstream task
 - **RAG for Knowledge-Intensive NLP Tasks**
 - REPLUG
- More comprehensive Interaction between language model and retriever
 - Self-RAG

RAG for Knowledge-Intensive NLP Tasks



- **Full Paper Title:** Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks
- **Authors:** Lewis et al. (FAIR, NYU, University College London)
- **Publication:** NeurIPS 2020
- **Link:** <https://arxiv.org/abs/2005.11401>

Terminology

Term	Definition
Knowledge-intensive tasks	Tasks that require external knowledge to answer
Parametric memory	Pre-trained model knowledge (i.e., model weights)
Non-parametric memory	External knowledge (e.g., retrieval systems)
Hybrid memory	Parametric + non-parametric (e.g., REALM)

Motivation: Hybrid Approach for seq2seq

- **Parametric:**
 - Difficult to **update or expand**
 - Cannot **inspect** or explain knowledge
 - May produce **hallucinations**
- **Non-parametric:**
 - Easy to **update or expand**
 - We can **inspect** directly
- **Hybrid:**
 - Only explored open-domain extractive QA
 - Not yet adapted for seq2seq models

Solution: RAG for seq2seq models

- **Goal:** Adapt hybrid methodology for seq2seq models
- **How:** A fine-tuning strategy named *retrieval-augmented generation* (RAG)
- **Architecture:**
 1. Pretrained seq2seq transformer: BART
 2. Pretrained neural retriever: Dense Passage Retriever (DPR)
 3. Dense vector index of Wikipedia
- **Assume:** Models already have trained ability to access knowledge

Introducing RAG

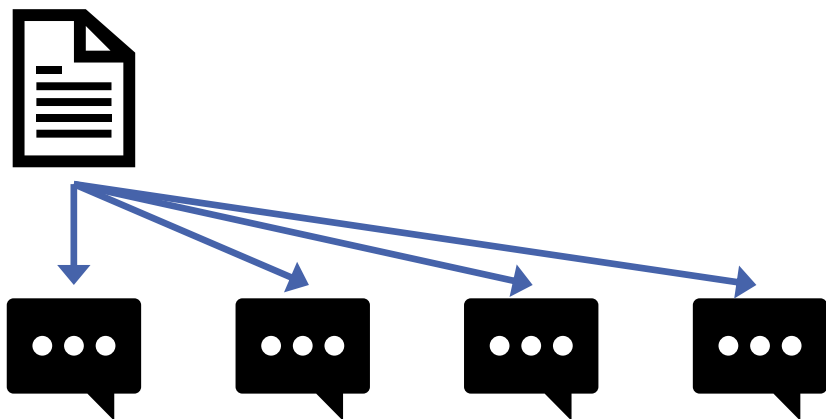
Idea: Combine **retriever** (nonparametric) and **generator** (parametric)

- **Retriever:** $p_{\eta}(z|x)$
 - Gives (top- k truncated) distribution over text passages z , given query x
- **Generator:** $p_{\theta}(y_i|x, z, y_{1:i-1})$
 - Gives next token y_i given previous tokens, query x , retrieved passage z

Two Types of RAG Models

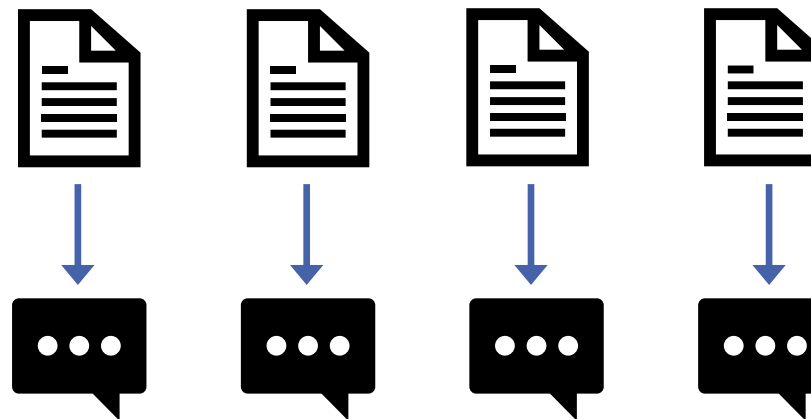
RAG-Sequence Model

Use **one** retrieved document to generate **entire** sequence



RAG-Token Model

Use **different** document to generate **each token** in sequence



Generators

- $p_{RAG-Sequence}(y|x) \approx \sum_{z \in top-k(p(\cdot|x))} p_{\eta}(z|x) \prod_i^N p_{\theta}(y_i|x, z, y_{1:i-1})$
 - Marginalized over each top- k document...
 - Generate probability for **entire output sequence**
- $p_{RAG-Token}(y|x) \approx \prod_i^N \sum_{z \in top-k(p(\cdot|x))} p_{\eta}(z|x) p_{\theta}(y_i|x, z, y_{1:i-1})$
 - For each token...
 - Marginalized over each each top- k document...
 - Generate distribution for **next output token**

Implementation: Encoding

- **Retriever:**
 - Uses DPR, which uses BERT to encode query and document
 - Similarity between query and document is computed using MIPS
- **Generator:**
 - Uses BART-large (400M parameters)
 - Input query and retrieved document are simply concatenated

Implementation: Decoding

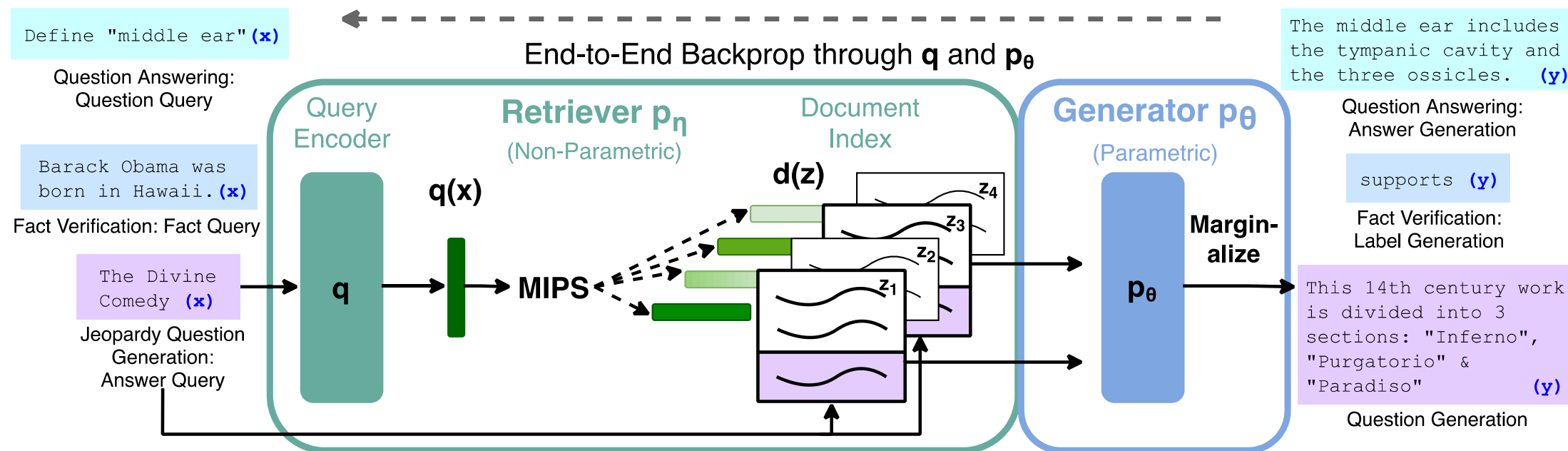
Question: How do we approximate $\operatorname{argmax}_y p(y|x)$?

- **RAG-Token:** Plug $p_\theta(y_i|x, z, y_{1:i-1})$ into standard beam decoder
- **RAG-Sequence:** Run beam search *for each* document z
 - $p(y|x)$ doesn't break into per-token likelihood
 - Therefore, it cannot be solved using a single beam search

Implementation: Fine-Tuning

- **Joint Fine-Tuning Update:**
 - **Update:** Retriever query encoder and generator
 - **Don't Update:** Retriever document encoder
 1. Expensive since document index must be periodically updated
 2. Authors find it isn't necessary for strong performance
 3. Aside: REALM does this during pre-training
- **Fine-Tuning Data:** A collection of pairs (x_j, y_j)
- **Objective:** Minimize $\sum_j -\log p(y_j|x_j)$ using SGD (via Adam)

Putting Everything Together



Experiment: Non-Parametric Setup

- **Data:**
 - Wikipedia dump from Dec 2018
 - 21M Wikipedia chunks (size of 100-words)
- **Indexing:**
 - Use document encoder (BERT) to compute embeddings
 - Embeddings put into MIPS index using FAISS
 - *Hierarchical Navigable Small World* used for fast retrieval
- **Retrieval:**
 - Training: Retrieve $k \in \{5,10\}$ documents
 - Testing: Set k using “dev data”

Experiment: The 4 Tasks

1. Open-Domain Question Answering
2. Abstractive Question Answering
3. Jeopardy Question Generation
4. Fact Verification

Task 1: Open-Domain QA

- **Task:** Answering objective **factual** questions
- **Data:**
 1. Natural Questions (NQ)
 2. TriviaQA (TQA)
 3. WebQuestions (WQ)
 4. CuratedTree (CT)
- **Setup:** Fine-tune using $(x, y) = (\text{question}, \text{answer})$ pairs
- **Baselines:**
 - Open-Book Models: Has access to external knowledge
 - Closed-Book Models: No access to external knowledge
- **Metrics:** Exact Match (EM) score

Task 1: RAG Has Strong QA Performance

- Why?
 - Can generate answers when answer not in documents
 - Even if answer isn't in documents, they can provide other useful info
- In general: RAG combine benefits of open + closed book

	Model	NQ	TQA	WQ	CT
Closed	T5-11B [52]	34.5	- /50.1	37.4	-
Book	T5-11B+SSM[52]	36.6	- /60.5	44.7	-
Open	REALM [20]	40.4	- / -	40.7	46.8
Book	DPR [26]	41.5	57.9 / -	41.1	50.6
	RAG-Token	44.1	55.2/66.1	45.5	50.0
	RAG-Seq.	44.5	56.8/ 68.0	45.2	52.2

Task 2: Abstractive QA

- **Task:** Natural language generation (NLG)
- **Data:** MSMARCO NLG task v2.1
 - A large dataset of questions
 - Each question has 10 gold search engine passages
 - Each question has full sentence answer annotated from passages
- **Setup:** Fine-tune using $(x, y) = (\text{question}, \text{answer})$ pairs
 - To force RAG to rely on parametric knowledge, ignore gold passages
- **Baselines:** BART, SotA
- **Metrics:** BLEU, Rouge-L

Task 2: RAG Approaches SotA on NLG

Model	Jeopardy		MSMARCO		FVR3 Label	FVR2 Acc.
	B-1	QB-1	R-L	B-1		
SotA	-	-	49.8*	49.9*	76.8	92.2*
BART	15.1	19.7	38.2	41.6	64.0	81.1
RAG-Tok.	17.3	22.2	40.1	41.5	72.5	<u>89.5</u>
RAG-Seq.	14.7	21.4	<u>40.8</u>	<u>44.2</u>		

- Consider:
 - SotA has access to gold passages, required for some answers
 - Not all questions are answerable from Wikipedia alone
- Nonetheless:
 - RAG outperforms BART
 - RAG approaches SotA

Task 3: Jeopardy Question Generation

Jeopardy question? A factual statement that prompts exact answer

- “In 1986 Mexico scored as the first country to host this international sports competition twice”
(Answer: “The World Cup”)
- **Task:** Generate Jeopardy questions conditioned on answer entities
- **Setup:** Fine-tune using $(x, y) = (\text{answer}, \text{question})$ pairs
- **Data:** SearchQA
- **Baselines:** BART
- **Metrics:** BLEU, Q-BLEU-1

Task 3: RAG-Token Performs Best

Model	Jeopardy		MSMARCO		FVR3	FVR2
	B-1	QB-1	R-L	B-1	Label	Acc.
SotA	-	-	49.8*	49.9*	76.8	92.2*
BART	15.1	19.7	38.2	41.6	64.0	81.1
RAG-Tok.	17.3	22.2	40.1	41.5	72.5	<u>89.5</u>
RAG-Seq.	14.7	21.4	<u>40.8</u>	<u>44.2</u>		

	Factuality	Specificity
BART better	7.1%	16.8%
RAG better	42.7%	37.4%
Both good	11.7%	11.8%
Both poor	17.7%	6.9%
No majority	20.8%	20.1%

- RAG-Token > RAG-Seq > BART

- RAG-Token can pull from *multiple* documents!
- RAG generators help to complete document-initiated generations

- Human evaluators prefer RAG

- Generations are more factual
- Generations are more specific

Task 4: Fact Verification

- **Task:** Classify whether a claim is true, false, or unverifiable
 - 3-Way Classification: Supports / Refutes / Not Enough Info
 - 2-Way Classification: Supports / Refutes
- **Data:** FEVER
 - A dataset of claims and evidence annotations for Wikipedia data
- **Setup:** Fine-tune using $(x, y) = (\text{claim}, \text{class})$ pairs
 - Each class mapped to unique output token
 - Similar to NLG, ignore evidence annotations
- **Baselines:** BART, SotA
- **Metrics:** Label accuracy

Task 4: RAG Scores Approach SotA

Model	Jeopardy		MSMARCO		FVR3	FVR2
	B-1	QB-1	R-L	B-1	Label	Acc.
SotA	-	-	49.8*	49.9*	76.8	92.2*
BART	15.1	19.7	38.2	41.6	64.0	81.1
RAG-Tok.	17.3	22.2	40.1	41.5	72.5	<u>89.5</u>
RAG-Seq.	14.7	21.4	<u>40.8</u>	<u>44.2</u>		

- Consider:

- SotA has access to gold evidence annotations
- SotA use complex pipelines, trained with supervision

- Nonetheless:

- RAG scores are close to SotA
- High overlap between RAG documents retrieved and gold evidence (not shown)

Outline

- Pre-training language models with the help of Retrieval
 - REALM
 - RETRO
- Jointly fine-tuning a retriever and language model for downstream task
 - RAG for Knowledge-Intensive NLP Tasks
 - REPLUG
- More comprehensive Interaction between language model and retriever
 - Self-RAG

REPLUG



- **Title:** REPLUG: Retrieval-Augmented Black Box Language Models
- **Authors:** Shi et al. (University of Washington, Stanford, KAIST, Meta AI)
- **Publication:** ArXiv 2023
- **Link:** <https://arxiv.org/abs/2301.12652>

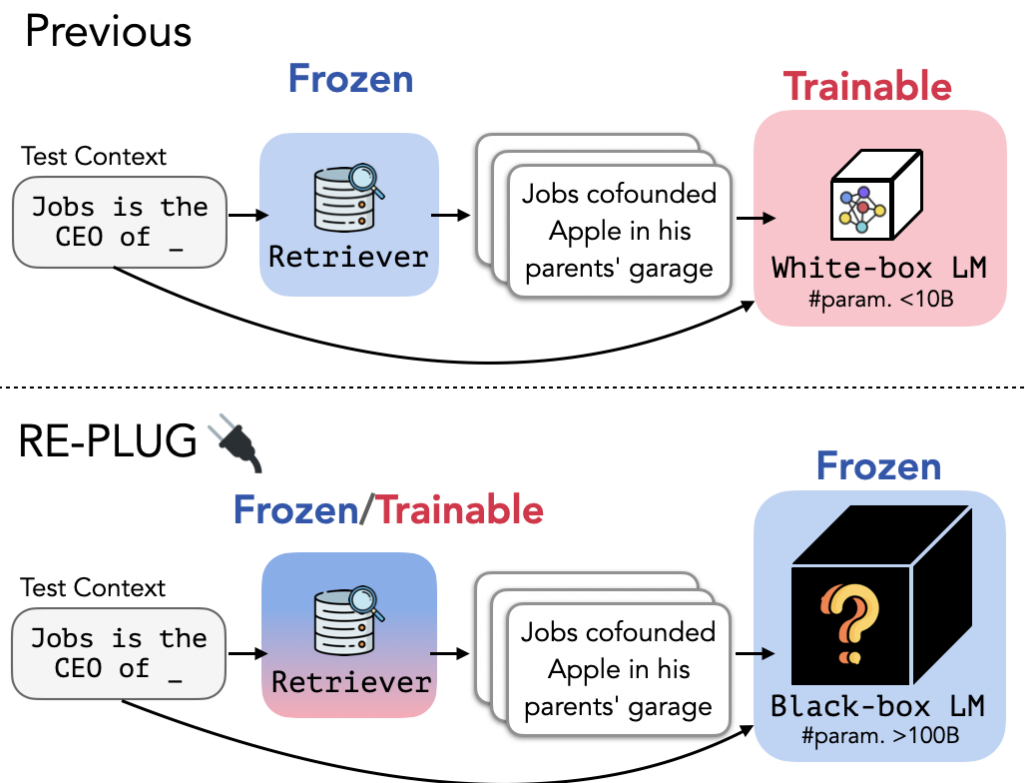
Motivation: RAG for Black-Box LMs

- **Issue:** Fine-tuning LMs with retrieval requires white-box access (i.e., access to the LM parameters)
 - To train the model
 - To index the datastore
- **In Practice:** Many LLMs are only accessible via API
 - Can't access model parameters!
 - Can't fine-tune!



Introducing REPLUG

- **Focus:** Retrieval-augmentation in the **black-box setting** (no access to model parameters)
 - LM is a black box
 - Retriever is optionally tunable
- **How does it work?**
 1. Get documents from retriever
 2. Prepend documents to LM input
 3. Feed input to LM



Implementation: Retrieval

- **Given:**
 - Corpus of m documents $D = \{d_1 \dots d_m\}$
 - Input context x
- **Output:**
 - The k documents in D that are most relevant to x , denoted $D' \subset D$
- **How:**
 - Precompute embedding for each document d (construct FAISS index)
 - Encode input x to obtain embedding
 - Find top- k most similar documents to x using *cosine* similarity function s

Hang On, Can We Fit k Documents?

- **Context Window:** The input text for a LM
- **Context Window Size:** Number of tokens a LM can process as input
- **Problem:** Context window may not fit k documents
 - The context window size varies by LM
 - The size of each document could vary
 - The number of retrieved documents k could vary

Solution: Ensemble Inference Scheme

- **Idea:** Do k separate LM predictions!
 1. Concatenate each $d \in D'$ with x
 2. Ensemble the output probabilities from k inference calls
- **Benefits:**
 - Inference calls can be parallelized!
 - Minimal overhead compared to single inference over k documents
 - Context window size need only fit x and largest document
- $p(y|x, D') = \sum_{d \in D'} p(y|d \circ x) \cdot \lambda(d, x)$
 - $\circ$ denotes concatenation
 - λ measures similarity of d and x
 - y is the next *token*

REPLUG LSR (LM-Supervised Retrieval)



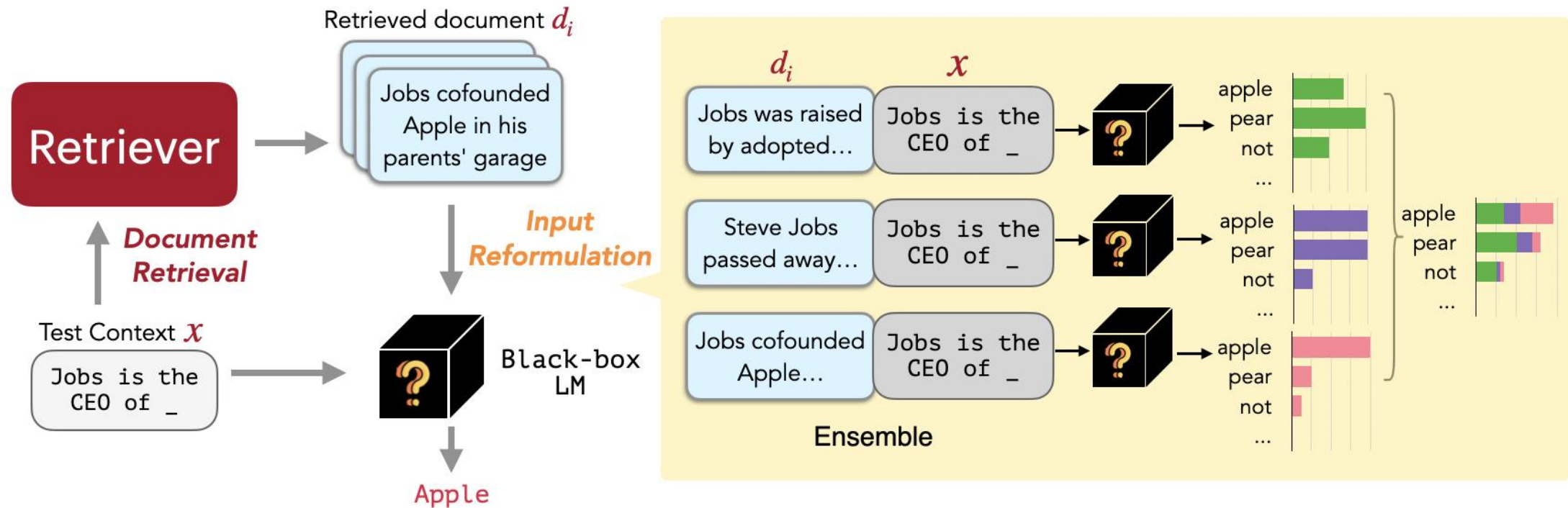
Idea: Train retriever to find documents that minimize LM perplexity

1. Retrieve k relevant documents $D' \subset D$
2. Compute retrieval likelihood of each document
 - $P_R(d|x) = \frac{e^{s(d,x)/\gamma}}{\sum_{d' \in D'} e^{s(d',x)/\gamma}}$
3. Compute LM likelihood of each document (y = ground truth)
 - $Q_{LM}(d|x, y) = \frac{e^{P_{LM}(y|d,x)/\beta}}{\sum_{d' \in D'} e^{P_{LM}(y|d',x)/\beta}}$
4. Minimize KL divergence between retriever and LM ($\mathcal{B}$ = contexts)
 - $\mathcal{L} = \frac{1}{|\mathcal{B}|} \sum_{x \in \mathcal{B}} KL(P_R(d|x) \parallel Q_{LM}(d|x, y))$

Implementation: Training

- **REPLUG:** No training, use off-the-shelf retriever (Contriever)
- **REPLUG LSR:** Use retriever (Contriever) + LM (GPT-3 Curie)
 - Sample training queries from Pile dataset (256 tokens x 800K sequences)
 - Query split into:
 - Input context x (128 tokens)
 - Ground truth continuation y (128 tokens)
 - Sample external documents from Pile (128 tokens x 36M documents)
 - Training queries and documents are verified for non-overlapping

Putting Everything Together



Experiments

1. Language Modeling
2. Massive Multi-task Language Understanding (MMLU)
3. Open-Domain QA

Task 1: Language Modeling

- **Data:** The Pile
- **Setup:**
 - REPLUG will use 128-token context window size
 - Retrieval corpus is 367M documents (each 128 tokens) sampled from The Pile
 - Retriever will retrieve top-10 documents
- **Baselines:** Test how much REPLUG improves...
 - GPT-2 (Small, Medium, Large, XL)
 - GPT-3 (Davinci, Curie, Babbage, Ada)
- **Metrics:** Bits per UTF-8 encoded byte (BPB)

Task 1: REPLUG Consistently Improves Language Modeling

Model			# Parameters	Original	+ REPLUG	Gain %	+ REPLUG LSR	Gain %
GPT-2	Small	117M		1.33	1.26	5.3	1.21	9.0
	Medium	345M		1.20	1.14	5.0	1.11	7.5
	Large	774M		1.19	1.15	3.4	1.09	8.4
	XL	1.5B		1.16	1.09	6.0	1.07	7.8
GPT-3 (black-box)	Ada	350M		1.05	0.98	6.7	0.96	8.6
	Babbage	1.3B		0.95	0.90	5.3	0.88	7.4
	Curie	6.7B		0.88	0.85	3.4	0.82	6.8
	Davinci	175B		0.80	0.77	3.8	0.75	6.3

Task 2: MMLU

- **Task / Data:** Multiple choice QA from MMLU dataset
 - Exam questions from 57 tasks spanning 4 categories:
 - Humanities, STEM, social sciences, other
- **Setup:**
 - Use Codex + REPLUG, evaluate in 5-shot setting
 - For each MMLU test query, retrieve and prepend 10 Wikipedia documents (Dec 2018 dump) and ensemble the output
- **Baselines:**
 - LLMs that are SotA for MMLU: Codex, PaLM, Flan-PaLM
 - RAG LMs: Atlas
- **Metrics:** Accuracy

Task 2: REPLUG Improves Codex

Model	# Parameters	Humanities	Social.	STEM	Other	All
Codex	175B	74.2	76.9	57.8	70.1	68.3
PaLM	540B	77.0	81.0	55.6	69.6	69.3
Flan-PaLM	540B	-	-	-	-	72.2
Atlas	11B	46.1	54.6	38.8	52.8	47.9
Codex + REPLUG	175B	76.0	79.7	58.8	72.1	71.4
Codex + REPLUG LSR	175B	76.5	79.9	58.9	73.2	71.8

Task 3: Open-Domain QA

- **Task:** Answering objective **factual** questions
- **Data:**
 1. Natural Questions (NQ)
 2. TriviaQA
- **Setup:**
 - Use Codex + REPLUG, evaluate in 16-shot setting
 - Wikipedia for retrieval, top-10 documents
- **Baselines:**
 - LLMs: Chinchilla, PaLM (both 64 shots), Codex (16 shots)
 - RAG LMs: RETRO, R2-D2, Atlas

Task 3: REPLUG Beats Previous RAG LMs



Model	NQ		TQA	
	Few-shot	Full	Few-shot	Full
Chinchilla	35.5	-	64.6	-
PaLM	39.6	-	-	-
Codex	40.6	-	73.6	-
RETRO [†]	-	45.5	-	-
R2-D2 [†]	-	55.9	-	69.9
Atlas [†]	42.4	60.4	74.5	79.8
Codex + Contriever _{cc} ²	44.2	-	76.0	-
Codex + REPLUG	44.7	-	76.8	-
Codex + REPLUG LSR	45.5	-	77.3	-

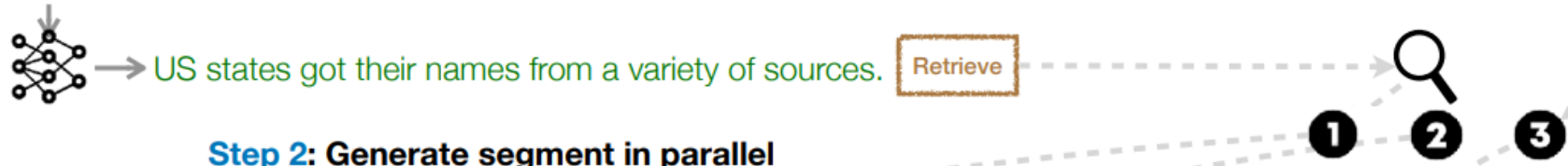
Outline

- Pre-training language models with the help of Retrieval
 - REALM
 - RETRO
- Jointly fine-tuning a retriever and language model for downstream task
 - RAG for Knowledge-Intensive NLP Tasks
 - REPLUG
- More comprehensive Interaction between language model and retriever
 - Self-RAG

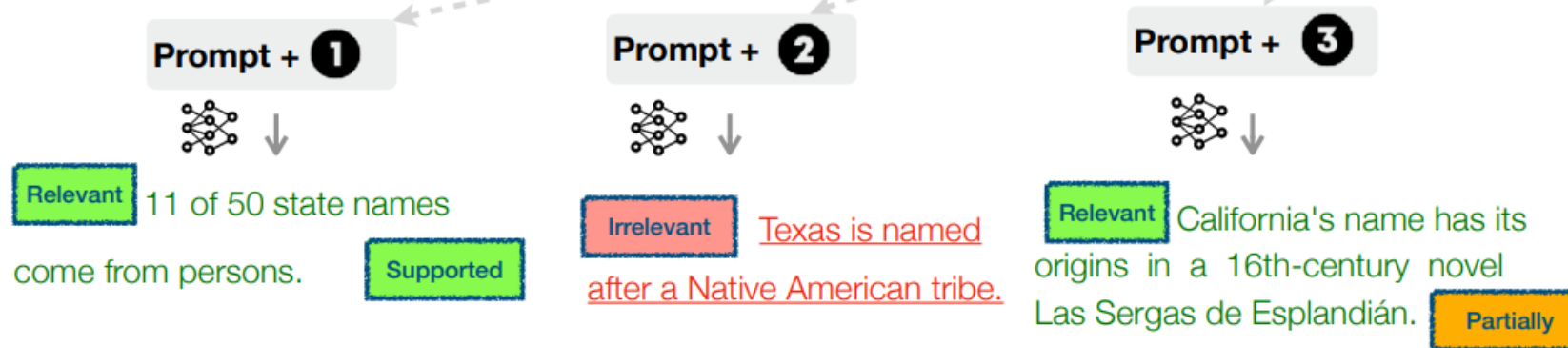
Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection

Prompt How did US states get their names?

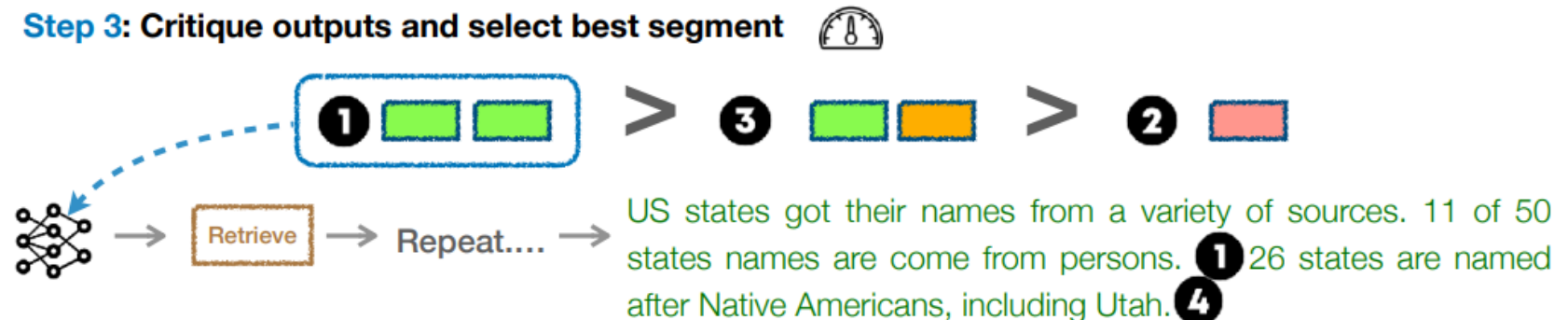
Step 1: Retrieve on demand



Step 2: Generate segment in parallel



Step 3: Critique outputs and select best segment



Problems of RAG Models

- Robustness to Irrelevant Contexts



Problems of RAG Models

- Attribution

On July 28th, 1942, an American actor was born. Who was she?

Sources

Famous People Born in
July 1942

 onthisday . 1

Tony Sirico - Wikipedia

 wikipedia . 2

Harrison Ford - Wikipedia

 wikipedia . 3



View 2 more

Answer

The American actor born on July 28th, 1942, is Sharon Gans ¹. She was an American stage and screen actress known for her roles in "Slaughterhouse-Five" and "Artists and Orphans: A True Drama" ¹.

<https://www.perplexity.ai/>

Main Components in Self-RAG

- Designed a special set of tokens called "*reflection tokens*", which enables the base LM to decide when to retrieve evidence and re-evaluate its own generation quality.

Type	Input	Output	Definitions
Retrieve	$x / x, y$	{yes, no, continue}	Decides when to retrieve with $\mathcal{R}$
ISREL	x, d	{ relevant , irrelevant}	d provides useful information to solve x .
ISSUP	x, d, y	{ fully supported , partially supported, no support}	All of the verification-worthy statement in y is supported by d .
ISUSE	x, y	{ 5 , 4, 3, 2, 1}	y is a useful response to x .

Table 1: Four types of reflection tokens used in SELF-RAG. Each type uses several tokens to represent its output values. The bottom three rows are three types of **Critique** tokens, and **the bold text** indicates the most desirable critique tokens. x, y, d indicate input, output, and a relevant passage, respectively.

Main Components in Self-RAG

- Generator and Critic
 - The generator generates next segment given the previous context.
 - The critic evaluates the retrieved evidence, generated segments, and overall response
 - The generator and critic can be seen as different roles of the same model (this is where the "self" comes from in "Self-RAG"); each role is invoked by specific reflection tokens.

Algorithm 1 SELF-RAG Inference

Require: Generator LM $\mathcal{M}$, Retriever $\mathcal{R}$, Large-scale passage collections $\{d_1, \dots, d_N\}$

- 1: **Input:** input prompt x and preceding generation $y_{<t}$, **Output:** next output segment y_t
 - 2: $\mathcal{M}$ predicts **Retrieve** given $(x, y_{<t})$
 - 3: **if** **Retrieve** == Yes **then**
 - 4: Retrieve relevant text passages $\mathbf{D}$ using $\mathcal{R}$ given (x, y_{t-1}) ▷ Retrieve
 - 5: $\mathcal{M}$ predicts **ISREL** given x, d and y_t given $x, d, y_{<t}$ for each $d \in \mathbf{D}$ ▷ Generate
 - 6: $\mathcal{M}$ predicts **ISSUP** and **ISUSE** given x, y_t, d for each $d \in \mathbf{D}$ ▷ Critique
 - 7: Rank y_t based on **ISREL**, **ISSUP**, **ISUSE** ▷ Detailed in Section 3.3
 - 8: **else if** **Retrieve** == No **then**
 - 9: $\mathcal{M}_{gen}$ predicts y_t given x ▷ Generate
 - 10: $\mathcal{M}_{gen}$ predicts **ISUSE** given x, y_t ▷ Critique
-

Self-RAG Inference Details

- Adaptive retrieval with threshold
 - Only when the probability of "Retrieve=YES" exceeds some threshold, the retrieval component will be triggered

Self-RAG Inference Details

- Adaptive retrieval with threshold
 - Only when the probability of "Retrieve=YES" exceeds some threshold, the retrieval component will be triggered
- Tree-decoding with critique tokens
 - Evaluate multiple retrieved passages in parallel
 - Conduct a segment-level beam search to obtain the top segment continuations at each timestamp and return the best sequence.
 - Segment selection

$$f(y_t, d, \boxed{\text{Critique}}) = p(y_t | x, d, y_{<t}) + \mathcal{S}(\boxed{\text{Critique}}), \text{ where}$$

$$\mathcal{S}(\boxed{\text{Critique}}) = \sum_{G \in \mathcal{G}} w^G s_t^G \text{ for } \mathcal{G} = \{\boxed{\text{ISREL}}, \boxed{\text{ISSUP}}, \boxed{\text{ISUSE}}\},$$

Self-RAG Example



Input Identify the two types of computer memory. (from gpt4_alpaca)

Output [**Retrieve**=Yes]<p>Computer memory Computer memory – In computing, memory refers to the computer hardware integrated circuits that store information for immediate use in a computer; it is synonymous with the term “primary storage”. Computer memory operates at a high speed, for example random-access memory (RAM), as a distinction from storage that provides slow-to-access information but offers higher capacities. If needed, contents of the computer memory can be transferred to secondary storage; a very common way of doing this is through a memory management technique called “virtual memory”. An archaic synonym for memory is store. The term “memory”, meaning “primary storage” or “main memory”, is<p>[**ISREL** =Relevant]**Primary** memory, also known as main memory, is the memory that the computer can directly access and manipulate. [**ISUP** =Fully Supported] [**Retrieve**=Yes]<p>Computer memory—often associated with addressable semiconductor memory, i.e. integrated circuits consisting of silicon-based transistors, used for example as primary storage but also other purposes in computers and other digital electronic devices. There are two main kinds of semiconductor memory, volatile and non-volatile. Examples of non-volatile memory are flash memory (used as secondary memory) and ROM, PROM, EPROM and EEPROM memory (used for storing firmware such as BIOS). Examples of volatile memory are primary storage, which is typically dynamic random-access memory (DRAM), and fast CPU cache memory, which is typically static random-access memory (SRAM) that is fast but energy-consuming, offering lower memory<p>[**ISREL** =Relevant]**Primary** memory can be accessed quickly, but it is volatile, meaning that the data is lost if the power is turned off.[**ISUP** =Partially Supported]...[**ISUSE** :5]

Self-RAG Data Collection

- The authors create supervised data by prompting GPT-4 to generate reflection tokens and then distill their knowledge into an in-house generator/critic.
- As different reflection token groups have their own definitions and input, they use different instruction prompts for them.
- Manual assessment reveals that GPT-4 reflection token predictions show high agreement with human evaluations.

Critic learning. After we collect training data $\mathcal{D}_{critic}$, we initialize $\mathcal{C}$ with a pre-trained LM and train it on $\mathcal{D}_{critic}$ using a standard conditional language modeling objective, maximizing likelihood:

$$\max_{\mathcal{C}} \mathbb{E}_{((x,y),r) \sim \mathcal{D}_{critic}} \log p_{\mathcal{C}}(r|x, y), \quad r \text{ for reflection tokens.} \quad (1)$$

Generator learning. We train the generator model $\mathcal{M}$ by training on the curated corpus augmented with reflection tokens $\mathcal{D}_{gen}$ using the standard next token objective:

$$\max_{\mathcal{M}} \mathbb{E}_{(x,y,r) \sim \mathcal{D}_{gen}} \log p_{\mathcal{M}}(y, r|x). \quad (2)$$

Unlike $\mathcal{C}$ training (Eq. 1), $\mathcal{M}$ learns to predict the target output as well as the reflection tokens. During training, we mask out the retrieved text chunks (surrounded by $\langle p \rangle$ and $\langle /p \rangle$ in Figure 2) for loss calculation and expand the original vocabulary $\mathcal{V}$ with a set of reflection tokens $\{\text{Critique}, \text{Retrieve}\}$.

Datasets and metrics

- Closed-set tasks (metrics: accuracy)
 - PubHealth: fact verification dataset about public health
 - ARC Challenge: multiple-choice reasoning dataset created from scientific exams
- Short-form QA (metrics: em)
 - PopQA and TriviaQA: Wikipedia-based open-domain QA datasets
- Long-form QA
 - ALCE-ASQA (Metrics: MAUVE, citation precision and recall): Citation evaluation
 - FactScore (Metrics: FactScore): Biography evaluation

Baselines

- Baselines without retrievals:
 - Llama2 (7B, 13B)
 - Alpaca(7B, 13B)
 - Llama2-chat (13B)
 - ChatGPT
 - CoVE (65B)
- Baselines with retrievals
 - RAG (Llama2, Alpaca)
 - Ret-ChatGPT and Ret-Llama2-chat
 - Perplexity.ai
 - SAIL
 - Toolformer

Self-RAG Results

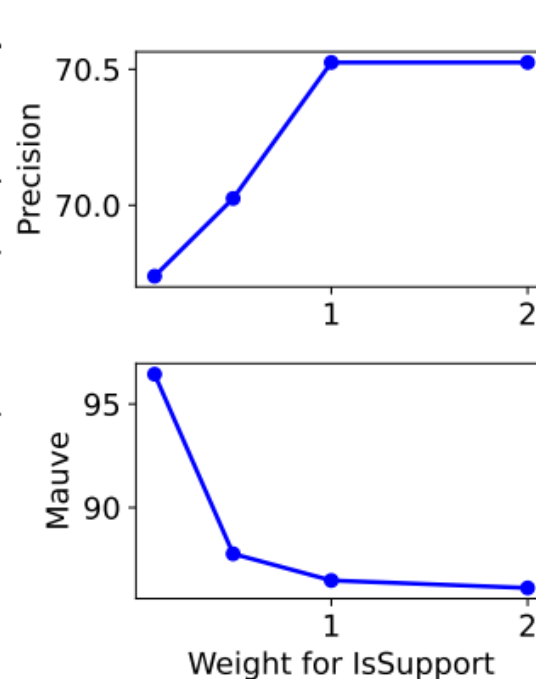


LM	Short-form		Closed-set		Long-form generations (with citations)					
	PopQA (acc)	TQA (acc)	Pub (acc)	ARC (acc)	Bio (FS)	(em)	(rg)	ASQA (mau)	(pre)	(rec)
<i>LMs with proprietary data</i>										
Llama2-c _{13B}	20.0	59.3	49.4	38.4	55.9	22.4	29.6	28.6	–	–
Ret-Llama2-c _{13B}	51.8	59.8	52.1	37.9	79.9	32.8	34.8	43.8	19.8	36.1
ChatGPT	29.3	74.3	70.1	75.3	71.8	35.3	36.2	68.8	–	–
Ret-ChatGPT	50.8	65.7	54.7	75.3	–	40.7	39.9	79.7	65.1	76.6
Perplexity.ai	–	–	–	–	71.2	–	–	–	–	–
<i>Baselines without retrieval</i>										
Llama2 _{7B}	14.7	30.5	34.2	21.8	44.5	7.9	15.3	19.0	–	–
Alpaca _{7B}	23.6	54.5	49.8	45.0	45.8	18.8	29.4	61.7	–	–
Llama2 _{13B}	14.7	38.5	29.4	29.4	53.4	7.2	12.4	16.0	–	–
Alpaca _{13B}	24.4	61.3	55.5	54.9	50.2	22.9	32.0	70.6	–	–
CoVE _{65B} *	–	–	–	–	71.2	–	–	–	–	–
<i>Baselines with retrieval</i>										
Toolformer* _{6B}	–	48.8	–	–	–	–	–	–	–	–
Llama2 _{7B}	38.2	42.5	30.0	48.0	78.0	15.2	22.1	32.0	2.9	4.0
Alpaca _{7B}	46.7	64.1	40.2	48.0	76.6	30.9	33.3	57.9	5.5	7.2
Llama2-FT _{7B}	48.7	57.3	64.3	65.8	78.2	31.0	35.8	51.2	5.0	7.5
SAIL* _{7B}	–	–	69.2	48.4	–	–	–	–	–	–
Llama2 _{13B}	45.7	47.0	30.2	26.0	77.5	16.3	20.5	24.7	2.3	3.6
Alpaca _{13B}	46.1	66.9	51.1	57.6	77.7	34.8	36.7	56.6	2.0	3.8
Our SELF-RAG _{7B}	54.9	66.4	72.4	67.3	81.2	30.0	35.7	74.3	66.9	67.8
Our SELF-RAG _{13B}	55.8	69.3	74.5	73.1	80.2	31.7	37.0	71.6	70.3	71.3

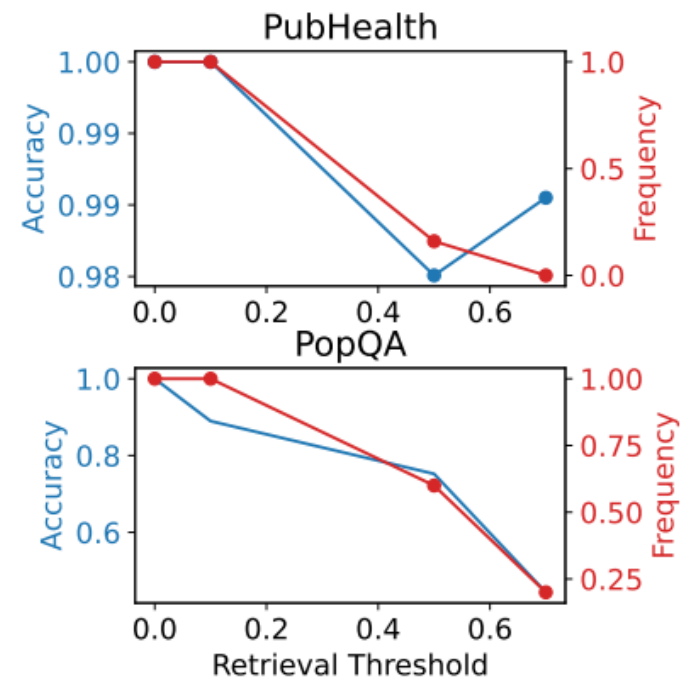
Self-RAG Analysis

	PQA (acc)	Med (acc)	AS (em)
SELF-RAG (50k)	45.5	73.5	32.1
<i>Training</i>			
No Retriever $\mathcal{R}$	43.6	67.8	31.0
No Critic $\mathcal{C}$	42.6	72.0	18.1
<i>Test</i>			
No retrieval	24.7	73.0	—
Hard constraints	28.3	72.6	—
Retrieve top1	41.8	73.1	28.6
Remove ISUP	44.1	73.2	30.6

(a) Ablation



(b) Customization



(c) Retrieval

Figure 3: **Analysis on SELF-RAG:** (a) **Ablation studies** for key components of SELF-RAG training and inference based on our 7B model. (b) **Effects of soft weights** on ASQA citation precision and Mauve (fluency). (c) **Retrieval frequency** and *normalized* accuracy on PubHealth and PopQA.

Self-RAG Discussion

- Connections to prior work on learning with critique.
 - Recent work incorporates additional critique (feedback) during training, e.g., RLHF via PPO. While PPO relies on separate reward models during training, we compute critique offline and directly insert them into the training corpus, where the generator LM is trained with a standard LM objective. This significantly reduces training costs compared to PPO.
- While Self-RAG shows significant improvements in terms of performance, factuality, and citation accuracy, it can still generate outputs that are not fully supported by the citations.

Q&A

- What types of tasks are not suitable for RAG?

Q&A

- What types of tasks are not suitable for RAG?
 - Lack of expert knowledge (i.e., retrieval coverage)
 - Doesn't require factual supports (i.e., writing a fiction)

Q&A

- What types of tasks are not suitable for RAG?
 - Lack of expert knowledge (i.e., retrieval coverage)
 - Doesn't require factual supports (i.e., writing a fiction)
- What's the down-side of Self-RAG and RePLUG?

- What types of tasks are not suitable for RAG?
 - Lack of expert knowledge (i.e., retrieval coverage)
 - Doesn't require factual supports (i.e., writing a fiction)
- What's the down-side of Self-RAG and RePLUG?
 - Latency: Processing multiple passages in parallel still bring up computational cost (depending on the GPU usage).

- What types of tasks are not suitable for RAG?
 - Lack of expert knowledge (i.e., retrieval coverage)
 - Doesn't require factual supports (i.e., writing a fiction)
- What's the down-side of Self-RAG and RePLUG?
 - Latency: Processing multiple passages in parallel still bring up computational cost (depending on the GPU usage).
- What are the pros and cons of different types of RAG?
 - Input text concatenation, cross-attention, output interpolation

- What types of tasks are not suitable for RAG?
 - Lack of expert knowledge (i.e., retrieval coverage)
 - Doesn't require factual supports (i.e., writing a fiction)
- What's the down-side of Self-RAG and RePLUG?
 - Latency: Processing multiple passages in parallel still bring up computational cost (depending on the GPU usage).
- What are the pros and cons of different types of RAG?
 - Input text concatenation, cross-attention, output interpolation
- Do we need RAG with long-context large language models?
 - Lost in the Middle: How Language Models Use Long Contexts